

```
$ docker compose up --build
```

Observability Simulator

A production-style telemetry pipeline built from scratch with Prometheus, Loki, Grafana and Alertmanager. Nine services, one command, the full path from instrumentation to dashboards to alerts.



Author: Pantelis Tsagkas

Repo: github.com/PantelisTsagkas/observability-simulator

Stack: Python 3.12, FastAPI, Prometheus, Loki, Grafana, Alertmanager, Docker Compose

FastAPI

Prometheus

Loki

Promtail

Grafana

Alertmanager

cAdvisor

Node Exporter

nginx

00 Why I built this

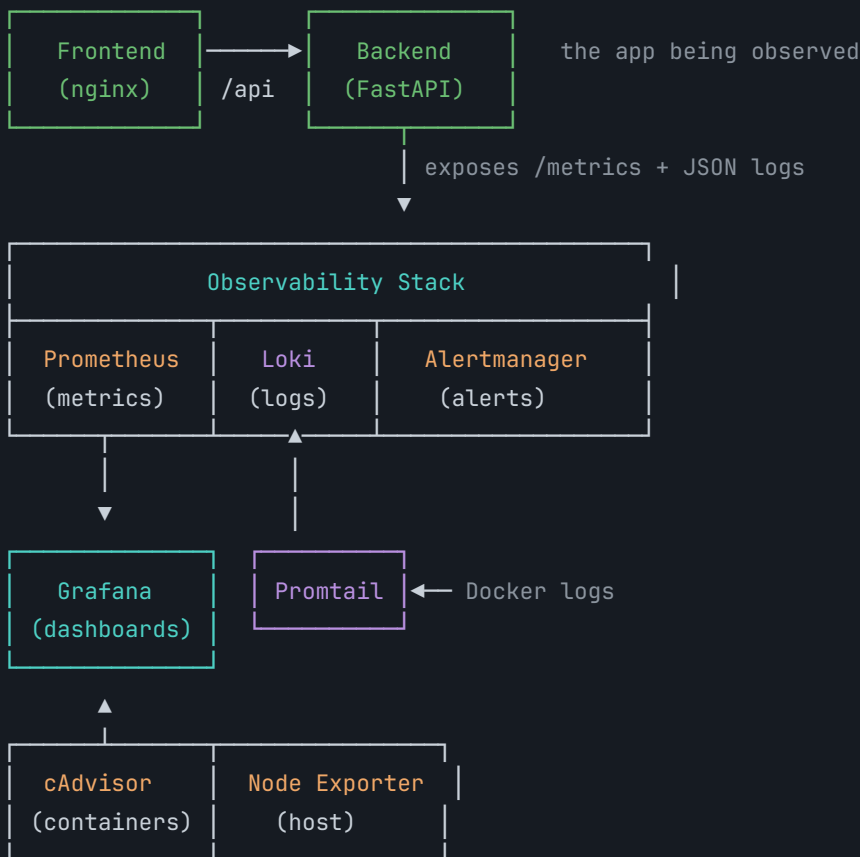
Most engineers learn observability the wrong way round. They click "enable metrics" on a managed service, get a dashboard handed to them, and never see what happens underneath. I wanted the opposite: build the whole pipeline myself so I understand every hop the data takes.

This project is a local sandbox that simulates a real service under load, then captures and visualises everything happening to it. I can trigger success responses, errors, slow requests, and CPU spikes on demand, then watch them flow through metrics collection, log aggregation, dashboards, and alerting in real time.

The goal was never a working demo for its own sake. It was to learn the open-source observability stack that real companies run, by wiring every component together by hand. Managed dashboards hide the mechanics. Here, nothing is hidden.

01 The architecture at a glance

The system splits into two halves. There is the application being watched, and there is the observability stack watching it. They share one Docker network and talk over plain HTTP.



Everything runs on a single Docker bridge network named `observability`. Inside it, containers reach each other by service name: Prometheus hits the backend at `http://backend:8000`, no IP addresses required. The ports exposed to my laptop exist only so I can open dashboards in a browser. No traffic leaves the host.

02 What each service does

The application layer



backend

app

:8000 • FastAPI

The thing being observed. A FastAPI service with endpoints that simulate real behaviour: instant success, 500 errors, slow responses, CPU burns, and random failures. It instruments itself with the `prometheus_client` library, exposing request counts, a latency histogram, and an error counter at `/metrics`. It also emits every request as a structured JSON log line via `structlog`. Important detail: it does not push anything anywhere. It just makes data available and writes to its own stdout.

**frontend**

app

:3000 · nginx

Serves the simulator UI and acts as a reverse proxy. When I click a button in the browser, nginx receives the request and forwards it to the backend over the internal network. The browser never talks to the backend directly, which mirrors how real deployments put a proxy in front of application servers.

The metrics pipeline

**prometheus**

metrics

:9090

A time-series database with a built-in scraper. Every 15 seconds it sends an HTTP GET to each target's `/metrics` endpoint, reads the numbers, and stores them indexed by time. This is the pull model: Prometheus goes and fetches data, nothing pushes to it. It keeps 7 days of history and continuously evaluates my alert rules against the data it holds.

**cadvisor**

metrics

:8080

Container Advisor. Runs with privileged access to Docker's internals and reports per-container CPU, memory, network, and disk usage. Without it I would only have the app's self-reported numbers. With it, I also know how much the backend container actually consumes at the OS level.

**node-exporter**

metrics

:9100

The same idea but for the host machine, not individual containers. It reads the Linux `/proc` and `/sys` virtual filesystems and exposes host-level CPU, memory, disk, and network metrics. It answers "is the machine healthy?" rather than "is this container healthy?"

The logging pipeline

**promtail**

logs

agent

A log shipping agent with read access to the Docker socket. It discovers running containers, tails their log streams in real time, and attaches labels to each line: `service`, `container`, and parsed JSON fields like `level` and `request_id`. It then ships those labelled entries to Loki over HTTP.

**loki**

logs

:3100

A log aggregation store, deliberately not a full-text search engine like Elasticsearch. It stores raw log lines cheaply and indexes only the labels Promtail attaches. That makes retrieval by label fast and cheap, full-text search slower. Grafana Labs designed it this way to solve the cost problem of running Elasticsearch at scale.

The alerting pipeline



alertmanager

alerts

:9093

When a Prometheus alert rule fires, Prometheus does not send the notification itself. It pushes the firing alert to Alertmanager, whose job is routing: which team to notify, on which channel, whether to group related alerts and suppress duplicates. In production this is where PagerDuty, Slack, or email get wired in. Keeping it separate from Prometheus is deliberate: alert routing is an operational concern, data collection is not.

The read layer



grafana

view

:3001

Stores no data of its own. It is purely query and visualisation. It is auto-provisioned with two data sources, Prometheus and Loki, plus pre-built dashboards loaded from JSON on startup. Metric panels send PromQL queries to Prometheus, log panels send LogQL queries to Loki. The auto-provisioning is what makes the whole stack reproducible with one command, no manual clicking.

03 How the data actually flows

The single most useful thing this project taught me is that metrics and logs travel in opposite directions, and understanding that distinction is the core of the whole stack.

Metrics: pulled

- 1 The backend exposes current values at `/metrics` and does nothing else. It is passive.
- 2 Prometheus reaches out every 15 seconds and scrapes the backend, cAdvisor, and Node Exporter.
- 3 It stores each reading against a timestamp in its time-series database.
- 4 It evaluates alert rules on every scrape: error rate, P95 latency, container CPU.
- 5 Grafana later queries Prometheus with PromQL to draw the graphs.

Logs: pushed

- 1 The backend writes JSON logs to stdout. Again, passive: it just writes.
- 2 Promtail tails the container's log stream through the Docker socket.
- 3 It labels each line and pushes it to Loki over HTTP.
- 4 Loki stores the lines and indexes the labels.
- 5 Grafana queries Loki with LogQL to show the logs alongside the metrics.

The key insight: the application is dumb on purpose. It does not know Prometheus, Loki, or Grafana exist. It exposes numbers and writes logs. Everything else is the observability stack reaching in to collect what it needs. This decoupling is why you can swap any component without touching the app, and why the same pattern scales from one container to a thousand.

Alerts: a chain of responsibility

Prometheus detects a problem and fires. Alertmanager decides who hears about it and how. Two separate jobs, two separate services. That separation is the whole design philosophy of this stack: small tools that each do one thing, connected by plain HTTP and a shared convention about what metrics look like.

04 This is what real companies run

This is not a toy stack. Prometheus plus Grafana plus Loki is the dominant open-source observability pattern in cloud-native engineering. Prometheus was created at SoundCloud in 2012 and became the second project to graduate from the Cloud Native Computing Foundation, right after Kubernetes itself. Below are organisations with publicly documented deployments, not marketing guesses.



Prometheus

SoundCloud

Uber

DigitalOcean

Red Hat

SUSE

Zerodha

Adform

Uber built and open-sourced its M3 platform to store Prometheus metrics at scale. Zerodha runs it for low-latency financial systems. Adform runs 10 instances at 2M+ time-series each. Source: CNCF case studies and graduation announcement.



Grafana

Bloomberg

NVIDIA

Microsoft

Salesforce

Anthropic

Grafana Labs reports more than 35 million users and 7,000+ paying customers, and names these organisations among them. Source: Grafana Labs company profile, 2025.



Loki

Paytm Insider

Qonto

Paytm Insider cut logging and monitoring costs by 75% and reduced mean-time-to-recovery from 30 to 10 minutes. Qonto ingests roughly 30 TB of compressed network logs per month through Loki. Source: Grafana Labs case studies.

Why that matters for me: what I learned here transfers directly. The skills are vendor-neutral. The same Prometheus instrumentation, the same PromQL, the same Loki labels run on AWS, GCP, Azure, or bare metal. CloudWatch knowledge only transfers within AWS. This transfers everywhere.

05 What I actually learned

Pull vs push is the foundation, not a detail

Before this I thought of monitoring as one vague thing. Now I understand that Prometheus pulls metrics on a schedule while Promtail pushes logs as they happen, and that this single distinction shapes how you design, scale, and debug the entire system.

Instrumentation is code you write, not a switch you flip

Exposing useful metrics means deciding what a counter, a gauge, and a histogram each measure, and choosing labels carefully. Too many label values blows up Prometheus memory. That tradeoff, cardinality, is a real production concern I now understand from having created the metrics myself.

Loki indexes labels, not content, and that choice is everything

Coming from the assumption that all log tools work like search engines, learning why Loki deliberately does not index full text reframed how I think about logging cost at scale. Cheap by design beats powerful by default when you are storing terabytes.

Separation of concerns is physical here

Prometheus detects, Alertmanager routes, Grafana displays, Loki stores, Promtail ships. Each is a separate process doing one job. Seeing that boundary drawn as actual containers made the Unix philosophy concrete in a way no diagram ever did.

Declarative infrastructure pays for itself immediately

The entire nine-service system, including dashboards and datasources, comes up reproducibly from config files with one command. Provisioning Grafana from JSON rather than clicking through its UI is the difference between a demo and something a team can actually maintain.

Health checks and dependency ordering are not optional

Grafana must not start before Prometheus is healthy. Promtail must not ship before Loki is ready. Encoding those dependencies in Docker Compose, with real health checks rather than naive ordering, taught me why orchestration tools exist at all.

06 Honest limitations and where this goes next

I want to be straight about what this does and does not prove. It runs locally on synthetic traffic. It does not demonstrate handling real production scale, Prometheus cardinality problems under genuine load, or Loki's chunk storage behaviour at volume.

The natural next step is deploying the same stack to a small cloud environment, a single EC2 instance or a lightweight k3s cluster, with real ingress traffic. That would show the identical architecture in a context where the scaling concerns become real, with Kubernetes replacing Docker Compose as the orchestrator. The point of building it locally first was to understand every piece before adding the complexity of a cluster on top.

One currency note: this project uses Promtail, which still works and is fully supported. Grafana Labs now considers Promtail feature-complete and points new deployments toward Grafana Alloy, its unified collector. Swapping Promtail for Alloy is a clean upgrade path and a sensible next iteration.

The takeaway: I did not learn observability by reading about it or clicking a managed dashboard into existence. I learned it by wiring nine services together until the data flowed end to end, and then breaking things on purpose to watch the alerts fire. That is the version of understanding I trust.